数据库设计与开发
Database Design & Development

计算机学院　单纯
sherryshan@bit.edu.cn
2024年3月

关系数据库设计理论
Design Theory for Relational Database

计算机学院　单纯
sherryshan@bit.edu.cn
2024年3月

内容概览

- 4.1 问题的提出
- 4.2 规范化
- 4.3 数据依赖的公理系统
- 4.4 模式的分解
- 4.5 小结

4.1 问题的提出（1）

- 数据库设计
  - 数据库概念设计（ER模型）
  - 数据库逻辑设计（规范化理论）
  - 数据库物理设计
- 关系数据库逻辑设计
  - 针对具体问题，如何构造一个适合于它的数据库模式
  - 数据库逻辑设计的工具——关系数据库的规范化理论

4.1 问题的提出（2）

- 4.1.1 概念回顾
- 4.1.2 关系模式的形式化定义
- 4.1.3 什么是数据依赖
- 4.1.4 关系模式的简化定义
- 4.1.5 数据依赖对关系模式的影响

4.1.1 概念回顾

- 关系：描述实体、属性、实体间的联系。
  - 从形式上看，它是一张二维表，是所涉及属性的笛卡尔积的一个子集
- 关系模式：用来定义关系
- 关系数据库：基于关系模型的数据库，利用关系来描述现实世界
  - 从形式上看，它由一组关系组成
- 关系数据库的模式：定义这组关系的关系模式的全体

4.1.2 关系模式的形式化定义

- 关系模式由五部分组成，即它是一个五元组

  R（U，D，DOM，F）

  R：　关系名

  U：　组成该关系的属性名集合

  D：　属性组U中属性所来自的域

  DOM：属性向域的映象集合

  F：　属性间数据的依赖关系集合

4.1.3 什么是数据依赖（1）

1. 完整性约束的表现形式
   - 限定属性取值范围：例如学生成绩必须在0-100之间
   - 定义属性值间的相互关连（主要体现于值的相等与否），这就是数据依赖，它是数据库模式设计的关键

4.1.3 什么是数据依赖（2）

2. 数据依赖
   - 是通过一个关系中属性间值的相等与否体现出来的数据间的相互关系
   - 是现实世界属性间相互联系的抽象
   - 是数据内在的性质
   - 是语义的体现

4.1.3 什么是数据依赖（3）

3. 数据依赖的类型
   - 函数依赖（Functional Dependency，简记为FD）
   - 多值依赖（Multivalued Dependency，简记为MVD）
   - 其他

4.1.4 关系模式的简化定义

- 关系模式R（U，D，DOM，F）简化为一个三元组

  R（U，F）

  当且仅当U上的一个关系r 满足F时，r称为关系模式 R（U，F）的一个关系

4.1.5 数据依赖对关系模式的影响（1）

例：建立一个描述学校的数据库

- 涉及的对象包括

  学生的学号（Sno）、所在系（Sdept）

  系主任姓名（Mname）、课程名（Cname）

  成绩（Grade）
- 假设学校的数据库模式由一个单一的关系模式Student构成：　Student <U、F>

  则，该关系模式的属性集合为U ＝｛ Sno，Sdept，Mname，Cname，Grade ｝

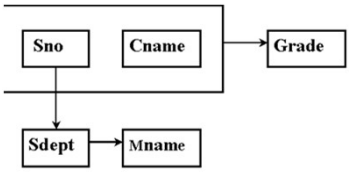
4.1.5 数据依赖对关系模式的影响（2）

- 学校数据库的语义
  - 一个系有若干学生， 一个学生只属于一个系
  - 一个系只有一名主任
  - 一个学生可以选修多门课程， 每门课程有若干学生选修
  - 每个学生所学的每门课程都有一个成绩
- 学校数据库.xlsx

4.1.5 数据依赖对关系模式的影响（3）

- 属性组U上的一组函数依赖F

  F ＝｛ Sno → Sdept，Sdept → Mname，（Sno, Cname）→ Grade ｝



关系模式Student<U,F>中存在的问题（1）

1. 数据冗余太大
   - 浪费大量的存储空间
     - 例：每一个系主任的姓名重复出现，重复次数与该系所有学生的所有课程成绩出现次数相同
2. 更新异常（Update Anomalies）
   - 数据冗余 ，更新数据时，维护数据完整性代价大。
     - 例：某系更换系主任后，系统必须修改与该系学生有关的每一个元组

关系模式Student<U,F>中存在的问题（2）

3. 插入异常（Insertion Anomalies）
   - 该插的数据插不进去
     - 例，如果一个系刚成立，尚无学生，我们就无法把这个系及其系主任的信息存入数据库
4. 删除异常（Deletion Anomalies）
   - 不该删除的数据不得不删
     - 例，如果某个系的学生全部毕业了， 我们在删除该系学生信息的同时，把这个系及其系主任的信息也丢掉了

课堂练习（1）

- 某学校图书馆要建一个图书数据库，其中借阅管理包括借书证号（CARDNO）、借书人姓名（NAME）、借书人所在单位（DEPT）、单位负责人（MN）、图书编号（BNO）、借阅日期等（DATE）信息。借阅图书登记可以用如下关系模式来描述

  BORROW（CARDNO，NAME，DEPT，MN，BNO，DATE）

课堂练习（2）

BORROW 表

CARDNO	NAME	DEPT	MN	BNO	DATE
R001	李晓鹏	计算机系	张宏军	TP31-125	2007.01.23
R001	李晓鹏	计算机系	张宏军	TP32-007	2006.11.12
R001	李晓鹏	计算机系	张宏军	TP12-233	2007.04.05
R002	王一鸣	计算机系	张宏军	TP51-211	2008.02.09
R002	王一鸣	计算机系	张宏军	TP31-254	2008.02.09
R003	刘明川	无线电系	范和平	TP23-126	2007.10.11
R003	刘明川	无线电系	范和平	TP23-023	2008.03.21
R003	刘明川	无线电系	范和平	TP25-045	2008.03.21

课堂练习（3）

- 存储异常
  - 数据冗余
  - 插入异常
  - 删除异常
  - 更新异常

数据冗余

- 借书人每借一本书，有关借书证号，借书人姓名及所在单位和单位负责人都要重复存储。大量的数据冗余不仅造成存储空间的浪费，而且存在着潜在的数据不一致

插入异常

- BORROW关系模式的关键字是由CARDNO和BNO组成的联合键，根据关系模型的实体完整性规则，关键字不能为空
- 如果一个人没有借书，就不能办理借阅手续，因为BNO为空。即，如果一个人没有借书，有关借书人的信息就不能存入数据库，这是不合理的

删除异常

- 当借书人归还所借的书后，需要从借阅关系中删除相关信息。但借阅人如果在某段时间把所借的书全部还清了，则在删除借书信息的同时，连同借阅人及所在单位的所有信息都一起从数据库中删去了，这是我们所不希望的

更新异常

- 如果借书人所在单位变了或单位负责人变了，需要修改该借书人在DEPT或MN属性上的值。但由于数据冗余，有关借书人所有元组中的单位或单位负责人的信息都要修改。这不仅增加了更新代价，而且存在着潜在的不一致性

课堂练习（4）

- 为什么会出现存储异常呢？
  - 因为在数据间存在着一定的依赖关系
    - 如借书关系中，借阅人、借阅人所在单位及单位负责人与借书证号之间以及借书证号、书号与借阅日期之间存在着依存关系
  - 但BORROW模式没有很好地反映这些关系
  - 更好的数据库模式是什么？

4.2 规范化（1）

- 规范化理论正是用来改造关系模式，通过分解关系模式来消除其中不合适的数据依赖，以解决插入异常、删除异常、更新异常和数据冗余问题

4.2 规范化（2）

- 4.2.1 函数依赖
- 4.2.2 键
- 4.2.3 范式
- 4.2.4 1NF
- 4.2.5 2NF

4.2 规范化（3）

- 4.2.6 3NF
- 4.2.7 BCNF
- 4.2.8 多值依赖与4NF
- 4.2.9 连接依赖与5NF
- 4.2.10 规范化小结

4.2.1 函数依赖

- 函数依赖
- 平凡函数依赖与非平凡函数依赖
- 完全函数依赖与部分函数依赖
- 传递函数依赖

函数依赖（1）

- 设R（U）是一个属性集U上的关系模式，X和Y是U的子集。若对于R（U）的任意一个可能的关系r，r中不可能存在两个元组在X上的属性值相等， 而在Y上的属性值不等， 则称 “X函数确定Y”或 “Y函数依赖于X”，记作X→Y
- Y=f（x），即若t_1 and t_2 in r（R），则 $t_1[x]=t_2[x] \Longrightarrow t_1[Y]=t_2[Y]$

函数依赖（2）

- 说明
  1. 函数依赖不是指关系模式R的某个或某些关系实例满足的约束条件，而是指R的所有关系实例均要满足的约束条件
  2. 函数依赖是语义范畴的概念。只能根据数据的语义来确定函数依赖
     - 例如“姓名→年龄”这个函数依赖只有在不允许有同名人的条件下成立。如果有相同名字的人，则“年龄”就不再函数依赖于“姓名”了

函数依赖（3）

- 说明（续）

3.数据库设计者可以对现实世界作强制的规定

- 例如在上例中，设计者可以强行规定不允许同名人出现，因而使函数依赖“姓名→年龄”成立。这样当插入某个元组时，这个元组上的属性值必须满足规定的函数依赖，若发现有同名人存在，则拒绝插入该元组

4.若X→Y，则X称为这个函数依赖的决定属性集（Determinant）

5.若X→Y，并且Y→X，则记为X ←→Y

6.若Y不函数依赖于X，则记为X $\not\rightarrow$Y

函数依赖（4）

- 例: Student（Sno，Sname，Ssex，Sage，Sdept）

假设不允许重名，则有

Sno →Ssex，Sno →Sage，Sno →Sdept，

Sno←→Sname，Sname→Ssex， Sname →Sage，

Sname →Sdept

但Ssex $\not\rightarrow$Sage

平凡函数依赖与非平凡函数依赖（1）

- 在关系模式R（U）中，对于U的子集X和Y，如果X→Y，但Y $\not\subseteq$ X，则称X→Y是非平凡的函数依赖；若X→Y，但Y $\subseteq$ X， 则称X→Y是平凡的函数依赖

例：在关系SC（Sno，Cno，Grade）中，

非平凡函数依赖： （Sno，Cno）→Grade

平凡函数依赖： （Sno，Cno）→ Sno

（Sno，Cno）→ Cno

平凡函数依赖与非平凡函数依赖（2）

- 于任一关系模式，平凡函数依赖都是必然成立的，它不反映新的语义，因此若不特别声明， 我们总是讨论非平凡函数依赖

完全函数依赖与部分函数依赖（1）

- 在关系模式R（U）中，如果X→Y，并且对于X的任何一个真子集X’，都有X’$\not\rightarrow$Y, 则称Y完全函数依赖于X，记作X$\xrightarrow{f}$ Y

若X→Y，但Y不完全函数依赖于X，则称Y部分函数依赖于X，记作X$\xrightarrow{p}$Y

完全函数依赖与部分函数依赖（2）

例：在关系SC（Sno，Cno，Grade）中，

由于：Sno $\not\rightarrow$Grade，Cno $\not\rightarrow$ Grade

因此：（Sno，Cno）$\xrightarrow{f}$Grade是完全函数依赖，（Sno，Cno）是决定属性集

由于：Sno →Sdept成立，且Sno是（Sno，Cno）的真子集

因此：（Sno，Cno）$\xrightarrow{p}$Sdept是部分函数依赖

传递函数依赖

- 在关系模式R（U）中，如果X→Y，Y→Z，且Y ⊆X，Y↛X，则称Z传递函数依赖于X
  - 注: 如果Y→X， 即X←→Y，则Z直接依赖于X
  - 例: 在关系Std（Sno，Sdept，Mname）中，有：

    Sno →Sdept，Sdept →Mname

    Mname传递函数依赖于Sno

课堂练习（1）

BORROW 表

CARDNO	NAME	DEPT	MN	BNO	DATE
R001	李晓鹏	计算机系	张宏军	TP31-125	2007.01.23
R001	李晓鹏	计算机系	张宏军	TP32-007	2006.11.12
R001	李晓鹏	计算机系	张宏军	TP12-233	2007.04.05
R002	王一鸣	计算机系	张宏军	TP51-211	2008.02.09
R002	王一鸣	计算机系	张宏军	TP31-254	2008.02.09
R003	刘明川	无线电系	范和平	TP23-126	2007.10.11
R003	刘明川	无线电系	范和平	TP23-023	2008.03.21
R003	刘明川	无线电系	范和平	TP25-045	2008.03.21

课堂练习（2）

- 借阅管理BORROW
  - 借书证号（CARDNO）
  - 借书人姓名（NAME）
  - 借书人所在单位（DEPT）
  - 单位负责人（MN）
  - 图书编号（BNO）
  - 借阅日期等（DATE）

课堂练习（3）

- BORROW中存在着哪些函数依赖？
- BORROW中存在着哪些完全函数依赖？
- BORROW中存在着哪些部分函数依赖？
- BORROW中存在着哪些传递函数依赖？

函数依赖

- CARDNO→NAME
- CARDNO→DEPT
- CARDNO→MN
- DEPT→MN
- （CARDNO，BNO）→DATE

完全函数依赖

- （CARDNO，BNO）$\xrightarrow{f}$DATE
- CARDNO$\xrightarrow{f}$NAME
- CARDNO$\xrightarrow{f}$DEPT
- CARDNO$\xrightarrow{f}$MN
- DEPT$\xrightarrow{f}$MN

部分函数依赖

- （CARDNO，BNO）$\xrightarrow{p}$NAME
- （CARDNO，BNO）$\xrightarrow{p}$DEPT
- （CARDNO，BNO）$\xrightarrow{p}$MN

传递函数依赖

- CARDNO→MN

4.2.2 键（1）

- 设K为关系模式R<U，F>中的属性或属性组合。若K$\xrightarrow{f}$ U，则K称为R的一个候选键（Candidate Key）。若关系模式R有多个候选键，则选定其中的一个做为主键（Primary Key）

4.2.2 键（2）

- 主属性与非主属性
  - 包含在任何一个候选键中的属性，称为主属性（Prime attribute）
  - 不包含在任何键中的属性称为非主属性（Nonprime attribute）或非键属性（Non-key attribute）
- 全键
  - 整个属性组是键，称为全键（All-key）

4.2.2 键（3）

- 例
  - 关系模式S（Sno，Sdept，Sage），单个属性Sno是键
  - 关系模式SC（Sno，Cno，Grade）中，（Sno，Cno）是键

4.2.2 键（4）

- 例

关系模式R（P，W，A）

P：演奏者　W：作品　A：听众

一个演奏者可以演奏多个作品

某一作品可被多个演奏者演奏

听众可以欣赏不同演奏者的不同作品

键为（P，W，A），即All-Key

4.2.2 键（5）

- 关系模式 *R* 中属性或属性组*X* 并非 *R*的键，但 *X* 是另一个关系模式的键，则称 *X* 是 *R* 的外键（Foreign Key）
  - 如在SC（Sno，Cno，Grade）中，Sno不是键，但Sno是关系模式S（Sno，Sdept，Sage）的键，则Sno是关系模式SC的外键
  - 主键与外键一起提供了表示关系间联系的手段

4.2.3 范式（1）

- 范式是符合某一种级别的关系模式的集合
- 关系数据库中的关系必须满足一定的要求。满足不同程度要求的为不同范式

4.2.3 范式（2）

- 范式的种类
  - 第一范式（1NF）
  - 第二范式（2NF）
  - 第三范式（3NF）
  - BC范式（BCNF）
  - 第四范式（4NF）
  - 第五范式（5NF）

4.2.3 范式（3）

- 各种范式之间存在联系

  $1NF \supset 2NF \supset 3NF \supset BCNF \supset 4NF \supset 5NF$
- 某一关系模式R为第n范式，可简记为R∈nNF
- 一个低一级范式的关系模式，通过模式分解可以转换为若干个高一级范式的关系模式的集合，这种过程就叫规范化

4.2.4 1NF（1）

- 1NF的定义
  - 如果一个关系模式R的所有属性都是不可分的基本数据项，则R∈1NF
- 一个给定关系和第一范式（1NF）的区别
  - 一个关系（二维表格）中的数据按照行和列的形式组织。表中的每个元组具有相同数目的属性个数，且每一个元组分量（属性值）具有统一的数据类型和长度；元组或属性的排列与顺序无关，元组的排列顺序调换或属性的排列顺序调换不影响关系的性质；每个元组必须通过一个属性或属性组（主键）唯一识别

4.2.4 1NF（2）

- 第一范式实际上是对关系增加了一个约束，即关系中元组的每个属性都只能取一个值。第一范式是对关系模式的基本要求，不满足第一范式的数据库就不是关系数据库
- 但是满足第一范式的关系模式并不一定是一个好的关系模式

4.2.5 2NF（1）

- 例
  - 关系模式 SLC（Sno，Sdept，Sloc，Cno，Grade），Sloc为学生住处，假设每个系的学生住在同一个地方

4.2.5 2NF（2）

- 函数依赖包括

  （Sno，Cno）$\xrightarrow{f}$Grade

  Sno → Sdept

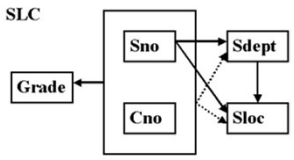
  （Sno，Cno）$\xrightarrow{p}$ Sdept

  Sno → Sloc

  （Sno，Cno）$\xrightarrow{p}$ Sloc

  Sdept → Sloc

4.2.5 2NF（3）



- SLC的键为（Sno，Cno）
- 结论
  - SLC满足第一范式
  - 非主属性Sdept和Sloc部分函数依赖于键（Sno, Cno）

4.2.5 2NF（4）

- SLC不是一个好的关系模式

  （1） 插入异常
  - 假设Sno＝95102，Sdept＝IS，Sloc＝N的学生还未选课，因课程号是主属性，因此该学生的信息无法插入SLC

  （2） 删除异常
  - 假定某个学生本来只选修了3号课程这一门课。现在因身体不适，他连3号课程也不选修了。因课程号是主属性，此操作将导致该学生信息的整个元组都要删除

4.2.5 2NF（5）

- SLC不是一个好的关系模式（续）

  （3） 数据冗余度大
  - 如果一个学生选修了10门课程，那么他的Sdept和Sloc值就要重复存储10次

  （4） 修改复杂
  - 例如学生转系，在修改此学生元组的Sdept值的同时，还可能需要修改住处（Sloc）。如果这个学生选修了K门课，则必须无遗漏地修改K个元组中全部Sdept、Sloc信息

4.2.5 2NF（6）

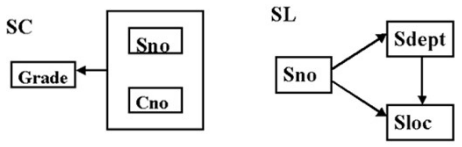
- 原因
  - Sdept、 Sloc部分函数依赖于键
- 解决方法
  - SLC分解为两个关系模式，以消除这些部分函数依赖

    SC（Sno， Cno， Grade）

    SL（Sno， Sdept， Sloc）

4.2.5 2NF（7）

- 函数依赖图



4.2.5 2NF（8）

- 在SC和SL中，非主属性都完全函数依赖于键了。从而使上述四个问题在一定程度上得到了一定的解决
  - （1）由于学生选修课程的情况与学生的基本情况是分开存储在两个关系中的，在SL关系中可以插入尚未选课的学生
  - （2）删除一个学生的所有选课记录，只是SC关系中没有关于该学生的记录了，SL关系中关于该学生的记录不受影响

4.2.5 2NF（9）

- 使上述四个问题在一定程度上得到了一定的解决（续）
  - （3）不论一个学生选多少门课程，他的Sdept和Sloc的值都只存储1次。这就大大降低了数据冗余
  - （4）学生转系只需改SL关系中该学生元组的Sdept值和Sloc值，由于Sdept、Sloc并未重复存储，因此减化了修改操作

4.2.5 2NF（10）

- 2NF的定义
  - 若关系模式R∈1NF，并且每一个非主属性都完全函数依赖于R的键，则R∈2NF
  - 在2NF中不存在非主属性对键的部分函数依赖
  - 例

    SLC（Sno，Sdept，Sloc，Cno，Grade）∈1NF

    SLC（Sno，Sdept，Sloc，Cno，Grade）∉2NF

    SC（Sno，Cno，Grade）∈ 2NF

    SL（Sno，Sdept，Sloc）∈ 2NF

4.2.5 2NF（11）

- 采用投影分解法将一个1NF的关系分解为多个2NF的关系，可以在一定程度上减轻原1NF关系中存在的插入异常、删除异常、数据冗余度大、修改复杂等问题
- 将一个1NF关系分解为多个2NF的关系，并不能完全消除关系模式中的各种异常情况和数据冗余

4.2.6 3NF（1）

- 例
  - 2NF关系模式SL（Sno，Sdept，Sloc）中函数依赖

    Sno→Sdept

    Sdept→Sno

    Sdept→Sloc

    Sloc传递函数依赖于Sno，即SL中存在非主属性对键的传递函数依赖

4.2.6 3NF（2）

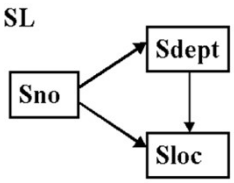
- SL关系存在的问题
  - （1）插入异常
    - 如果某个系因种种原因（例如刚刚成立），目前暂时没有在校学生，我们就无法把这个系的信息存入数据库
  - （2）删除异常
    - 如果某个系的学生全部毕业了，我们在删除该系学生信息的同时，把这个系的信息也丢掉了

4.2.6 3NF（3）

- SL关系存在的问题（续）
  - （3）数据冗余度大
    - 每一个系的学生都住在同一个地方，关于系的住处的信息却重复出现，重复次数与该系学生人数相同
  - （4）修改复杂
    - 当学校调整学生住处时，由于关于每个系的住处信息是重复存储的，修改时必须同时更新该系所有学生的Sloc属性值
  - 所以，SL仍不是一个好的关系模式

4.2.6 3NF（4）

- 函数依赖图



4.2.6 3NF（5）

- 原因
  - Sloc传递函数依赖于Sno
- 解决方法
  - 采用投影分解法，把SL分解为两个关系模式，以消除传递函数依赖

    SD（Sno， Sdept）

    DL（Sdept， Sloc）

    SD的键为Sno， DL的键为Sdept
  - 分解后的关系模式SD与DL中不再存在传递依赖

4.2.6 3NF（6）

- SD的键为Sno， DL的键为Sdept

4.2.6 3NF（7）

- 在分解后的关系模式中既没有非主属性对键的部分函数依赖也没有非主属性对键的传递函数依赖，在一定程度上解决了上述四个问题

4.2.6 3NF（8）

（1）DL关系中可以插入无在校学生的系的信息

（2）某个系的学生全部毕业了，只是删除SD关系中的相应元组，DL关系中关于该系的信息仍存在

（3）关于系的住处的信息只在DL关系中存储一次

（4）当学校调整某个系的学生住处时，只需修改DL关系中一个相应元组Sloc属性值

4.2.6 3NF（9）

- 3NF的定义
  - 关系模式R<U，F> 中若不存在这样的键X、属性组Y及非主属性Z（$Z \not\subseteq Y$），使得X→Y，$Y \not\to X$，Y→Z，成立，则称R<U，F> ∈ 3NF
  - 例

    SL（Sno, Sdept, Sloc）∈ 2NF

    SL（Sno, Sdept, Sloc）$\notin$ 3NF

    SD（Sno， Sdept） ∈ 3NF

    DL（Sdept， Sloc）∈ 3NF

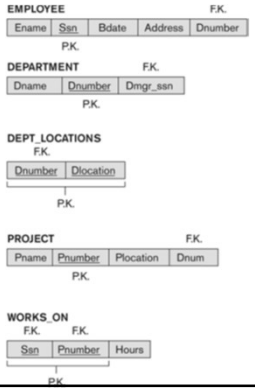
4.2.6 3NF（10）

- 第三范式是在第二范式的基础上对关系又增加了一个约束，就是关系中每一个非主属性必须只依赖于主键。换句话说，就是在第二范式的基础上，消除非主属性对主键的传递函数依赖可达到3NF
- 若R∈3NF，则R的每一个非主属性既不部分函数依赖于候选键也不传递函数依赖于候选键
- 如果R∈3NF，则R也是2NF

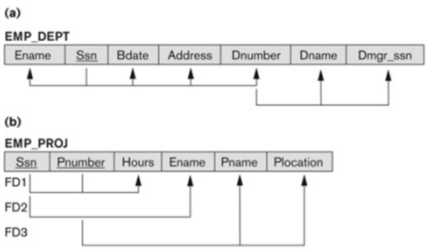
4.2.6 3NF（11）

- 采用投影分解法将一个2NF的关系分解为多个3NF的关系，可以在一定程度上解决原2NF关系中存在的插入异常、删除异常、数据冗余度大、修改复杂等问题
- 将一个2NF关系分解为多个3NF的关系后，并不能完全消除关系模式中的各种异常情况和数据冗余

A simplified COMPANY relational database schema



Two relation schemas suffering from update anomalies



Example States for EMP_DEPT and EMP_PROJ

EMP_DEPT

Ename	Ssn	Bdate	Address	Dnumber	Dname	Dmgr_ssn
Smith, John B.	123456789	1965-01-09	731 Fondren, Houston, TX	5	Research	333445555
Wong, Franklin T.	333445555	1955-12-08	638 Voss, Houston, TX	5	Research	333445555
Zelaya, Alicia J.	999887777	1968-07-19	3321 Castle, Spring, TX	4	Administration	987654321
Wallace, Jennifer S.	987654321	1941-06-20	291 Berry, Bellaire, TX	4	Administration	987654321
Narayan, Ramesh K.	666884444	1962-09-15	975 FireOak, Humble, TX	5	Research	333445555
English, Joyce A.	453453453	1972-07-31	5631 Rice, Houston, TX	5	Research	333445555
Jabbar, Ahmad V.	987987987	1969-03-29	980 Dallas, Houston, TX	4	Administration	987654321
Borg, James E.	888665555	1937-11-10	450 Stone, Houston, TX	1	Headquarters	888665555

Redundancy Redundancy

EMP_PROJ

Ssn	Pnumber	Hours	Ename	Pname	Plocation
123456789	1	32.5	Smith, John B.	ProductX	Bellaire
123456789	2	7.5	Smith, John B.	ProductY	Sugarland
666884444	3	40.0	Narayan, Ramesh K.	ProductZ	Houston
453453453	1	20.0	English, Joyce A.	ProductX	Bellaire
453453453	2	20.0	English, Joyce A.	ProductY	Sugarland
333445555	2	10.0	Wong, Franklin T.	ProductY	Sugarland
333445555	3	10.0	Wong, Franklin T.	ProductZ	Houston
333445555	10	10.0	Wong, Franklin T.	Computerization	Stafford
333445555	20	10.0	Wong, Franklin T.	Reorganization	Houston
999887777	30	30.0	Zelaya, Alicia J.	Newbenefits	Stafford
999887777	10	10.0	Zelaya, Alicia J.	Computerization	Stafford
987987987	10	35.0	Jabbar, Ahmad V.	Computerization	Stafford
987987987	30	5.0	Jabbar, Ahmad V.	Newbenefits	Stafford
987654321	30	20.0	Wallace, Jennifer S.	Newbenefits	Stafford
987654321	20	15.0	Wallace, Jennifer S.	Reorganization	Houston
888665555	20	Null	Borg, James E.	Reorganization	Houston

Normalization into 1NF

(a)
DEPARTMENT

Dname	Dnumber	Dmgr_ssn	Dlocations

(b)
DEPARTMENT

Dname	Dnumber	Dmgr_ssn	Dlocations
Research	5	333445555	{Bellaire, Sugarland, Houston}
Administration	4	987654321	{Stafford}
Headquarters	1	888665555	{Houston}

(c)
DEPARTMENT

Dname	Dnumber	Dmgr_ssn	Dlocation
Research	5	333445555	Bellaire
Research	5	333445555	Sugarland
Research	5	333445555	Houston
Administration	4	987654321	Stafford
Headquarters	1	888665555	Houston

Normalization nested relations into 1NF

(a)
EMP_PROJ

		Projs	
Ssn	Ename	Pnumber	Hours

(b)
EMP_PROJ

Ssn	Ename	Pnumber	Hours
123456789	Smith, John B.	1	32.5
		2	7.5
666884444	Narayan, Ramesh K.	3	40.0
453453453	English, Joyce A.	1	20.0
		2	20.0
333445555	Wong, Franklin T.	2	10.0
		3	10.0
		10	10.0
		20	10.0
999887777	Zelaya, Alicia J.	30	30.0
		10	10.0
987987987	Jabbar, Ahmad V.	10	35.0
		30	5.0
987654321	Wallace, Jennifer S.	30	20.0
		20	15.0
888665555	Borg, James E.	20	NULL

(c)
EMP_PROJ1

Ssn	Ename

EMP_PROJ2

Ssn	Pnumber	Hours

Normalizing into 2NF and 3NF

(a)
EMP_PROJ

Ssn	Pnumber	Hours	Ename	Pname	Plocation

FD1
FD2
FD3

2NF Normalization

EP1

Ssn	Pnumber	Hours

FD1

EP2

Ssn	Ename

FD2

EP3

Pnumber	Pname	Plocation

FD3

(b)
EMP_DEPT

Ename	Ssn	Bdate	Address	Dnumber	Dname	Dmgr_ssn

3NF Normalization

ED1

Ename	Ssn	Bdate	Address	Dnumber

ED2

Dnumber	Dname	Dmgr_ssn

4.2.7 BCNF（1）

- BCNF（Boyce Codd Normal Form）是由Boyce和Codd提出的，比3NF更进了一步。通常认为BCNF是修正的第三范式，所以有时也称为扩充的第三范式
- 关系模式R<U，F>∈1NF。若X→Y且Y ⊆ X时X必含有键，则R<U，F> ∈BCNF。
  - 等价于：每一个决定属性因素都包含键

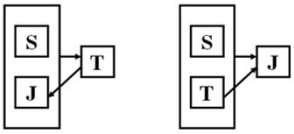
4.2.7 BCNF（2）

- 若R∈BCNF
  - 所有非主属性对每一个键都是完全函数依赖
  - 所有的主属性对每一个不包含它的键，也是完全函数依赖
  - 没有任何属性完全函数依赖于非键的任何一组属性

$$R \in BCNF \underset{\text{不必要}}{\overset{\text{充分}}{\rightleftarrows}} R \in 3NF$$

4.2.7 BCNF（3）

- 在关系模式STJ（S，T，J）中，S表示学生，T表示教师，J表示课程
  - 函数依赖
    - （S，J）→T
    - （S，T）→J
    - T→J



STJ中的函数依赖

4.2.7 BCNF（4）

- STJ∈3NF
  - 没有任何非主属性对键传递依赖或部分依赖
    - （S，J）和（S，T）都可以作为候选键
    - S、T、J都是主属性
- STJ∈BCNF
  - T是决定因素，T不包含键
    - T→J，T是决定属性集，可是T只是主属性，它既不是候选键也不包含侯选键

4.2.7 BCNF（5）

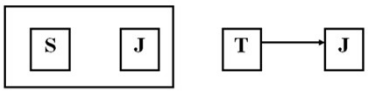
- 存在的问题
  - （1）插入异常
    - 如果某个学生刚刚入校，尚未选修课程，则因受主属性不能为空的限制，有关信息无法存入数据库中
    - 如果某个教师开设了某门课程，但尚未有学生选修，则有关信息也无法存入数据库中
  - （2）删除异常
    - 如果选修过某门课程的学生全部毕业了，在删除这些学生元组的同时，相应教师开设该门课程的信息也同时丢掉了

4.2.7 BCNF（6）

- 存在的问题（续）
  - （3）数据冗余度大
    - 虽然一个教师只教一门课，但每个选修该教师该门课程的学生元组都要记录这一信息
  - （4）修改复杂
    - 某个教师开设的某门课程改名后，所有选修了该教师该门课程的学生元组都要进行相应修改
  - STJ仍不是一个理想的关系模式

4.2.7 BCNF（7）

- 解决方法
  - 将STJ分解为二个关系模式
    - ST（S，T）∈ BCNF
    - TJ（T，J）∈ BCNF



ST　　TJ

  - 没有任何属性对键的部分函数依赖和传递函数依赖

4.2.7 BCNF（8）

- 在分解后的关系模式中没有任何属性对键的部分函数依赖和传递函数依赖。它解决了上述四个问题
  - （1）ST关系中可以存储尚未选修课程的学生。
    - TJ关系中可以存储所开设课程但尚未有学生选修的教师信息
  - （2）选修过某门课程的学生全部毕业了，只是删除ST关系中的相应元组，不会影响TJ关系中相应教师开设该门课程的信息

4.2.7 BCNF（9）

- 它解决了上述四个问题（续）
  - （3）关于每个教师开设课程的信息只在TJ关系中存储一次
  - （4）某个教师开设的某门课程改名后，只需修改TL关系中的一个相应元组即可

4.2.7 BCNF（10）

- 3NF与BCNF的关系
  - 如果关系模式R∈BCNF，必定有R∈3NF

$$R \in BCNF \underset{\text{不必要}}{\overset{\text{充分}}{\rightleftarrows}} R \in 3NF$$

  - 如果R∈3NF，且R只有一个候选键，则R必属于BCNF

$$R \in BCNF \underset{\text{必要}}{\overset{\text{充分}}{\rightleftarrows}} R \in 3NF$$

4.2.7 BCNF（11）

- BCNF的关系模式所具有的性质
  1. 所有非主属性都完全函数依赖于每个候选键
  2. 所有主属性都完全函数依赖于每个不包含它的候选键
  3. 没有任何属性完全函数依赖于非键的任何一组属性

4.2.8 多值依赖与4NF（1）

- 学校中某一门课程由多个教师讲授，他们使用相同的一套参考书。每个教员可以讲授多门课程，每种参考书可以供多门课程使用
  - 关系模式Teaching（C，T，B）
  - 课程C、教师T 和 参考书B

4.2.8 多值依赖与4NF（2）

- 非规范化关系示例

课 程 C	教 员 T	参 考 书 B
物理	李 勇 王 军	普通物理学 光学原理 物理习题集
数学	李 勇 张 平	数学分析 微分方程 高等代数
计算数学	张 平 周 峰	数学分析

4.2.8 多值依赖与4NF（3）

- 用二维表表示Teaching

课程C	教员T	参考书B
物 理	李 勇	普通物理学
物 理	李 勇	光学原理
物 理	李 勇	物理习题集
物 理	王 军	普通物理学
物 理	王 军	光学原理
物 理	王 军	物理习题集
数 学	李 勇	数学分析
数 学	李 勇	微分方程
数 学	李 勇	高等代数
数 学	张 平	数学分析
数 学	张 平	微分方程
数 学	张 平	高等代数
…	…	…

4.2.8 多值依赖与4NF（4）

- Teaching ∈BCNF
  - Teaching具有唯一候选键（C，T，B），即全键
- Teaching模式中存在的问题
  （1）数据冗余度大：有多少名任课教师，参考书就要存储多少次

4.2.8 多值依赖与4NF（5）

- Teaching模式中存在的问题（续）
  （2）插入操作复杂：当某一课程增加一名任课教师时，该课程有多少本参照书，就必须插入多少个元组
  - 例如物理课增加一名教师刘关，需要插入两个元组
    （物理，刘关，普通物理学）
    （物理，刘关，光学原理）

4.2.8 多值依赖与4NF（6）

- Teaching模式中存在的问题（续）
  （3）删除操作复杂：某一门课要去掉一本参考书，该课程有多少名教师，就必须删除多少个元组
  （4）修改操作复杂：某一门课要修改一本参考书，该课程有多少名教师，就必须修改多少个元组

4.2.8 多值依赖与4NF（7）

- 原因
  - 参考书的取值和教师的取值是彼此毫无关系的，都只取决于课程名
  - 存在多值依赖

4.2.8 多值依赖与4NF（8）

- 设R（U）是一个属性集U上的一个关系模式， X、 Y和Z是U的子集，并且Z＝U－X－Y，多值依赖 X→→Y成立当且仅当对R的任一关系r，r在（X，Z）上的每个值对应一组Y的值，这组值仅仅决定于X值而与Z值无关
  - 例 Teaching（C, T, B）
  - 对于C的每一个值，T有一组值与之对应，而不论B取何值

4.2.8 多值依赖与4NF（9）

- 多值依赖的另一个等价的形式化的定义
  - 在R（U）的任一关系r中，如果存在元组t，s使得t[X]=s[X]，那么就必然存在元组 w，v∈ r，（w，v可以与s，t相同），使得w[X]=v[X]=t[X]，而w[Y]=t[Y]，w[Z]=s[Z]，v[Y]=s[Y]，v[Z]=t[Z]（即交换s，t元组的Y值所得的两个新元组必在r中），则Y多值依赖于X，记为X→→Y。 这里，X，Y是U的子集，Z=U-X-Y

4.2.8 多值依赖与4NF（10）

- 平凡多值依赖和非平凡的多值依赖
  - 若X→→Y，而Z＝φ，则称
    X→→Y为平凡的多值依赖
  - 否则称X→→Y为非平凡的多值依赖

4.2.8 多值依赖与4NF（11）

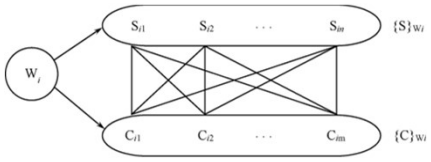
- 关系模式WSC（W，S，C）
  - W表示仓库，S表示保管员，C表示商品
  - 假设每个仓库有若干个保管员，有若干种商品
  - 每个保管员保管所在的仓库的所有商品
  - 每种商品被所有保管员保管

4.2.8 多值依赖与4NF（12）

W	S	C
W1	S1	C1
W1	S1	C2
W1	S1	C3
W1	S2	C1
W1	S2	C2
W1	S2	C3
W2	S3	C4
W2	S3	C5
W2	S4	C4
W2	S4	C5

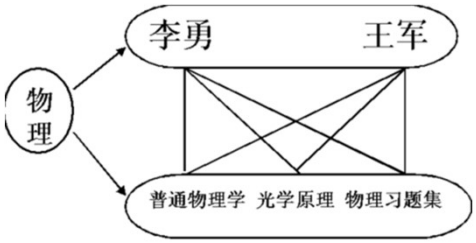
4.2.8 多值依赖与4NF（13）

- 用下图表示这种对应
  W→→S且W→→C



4.2.8 多值依赖与4NF（14）

- 多值依赖的性质
  （1）多值依赖具有对称性
  若X→→Y，则X→→Z，其中Z＝U－X－Y



4.2.8 多值依赖与4NF（15）

- 多值依赖的性质（续）
  （2）多值依赖具有传递性
  若X→→Y，Y→→Z， 则X→→Z –Y
  （3）函数依赖是多值依赖的特殊情况。
  若X→Y，则X→→Y
  （4）若X→→Y，X→→Z，则X→→Y∪Z
  （5）若X→→Y，X→→Z，则X→→Y∩Z
  （6）若X→→Y，X→→Z，则X→→Y-Z，
  X→→Z -Y

4.2.8 多值依赖与4NF（16）

- 多值依赖与函数依赖的区别
  （1）多值依赖的有效性与属性集的范围有关
  （2）
  - 若函数依赖X→Y在R（U）上成立，则对于任何Y'⊂Y均有X→Y' 成立
  - 多值依赖X→→Y若在R（U）上成立，不能断言对于任何Y'⊂Y有X→→Y' 成立

4.2.8 多值依赖与4NF（17）

- 关系模式R<U，F>∈1NF，如果对于R的每个非平凡多值依赖X→→Y（Y⊈X），X都含有候选键，则R∈4NF
- 如果R ∈ 4NF， 则R ∈ BCNF
  - 不允许有非平凡且非函数依赖的多值依赖
  - 允许的非平凡多值依赖是函数依赖

4.2.8 多值依赖与4NF（18）

- 例： Teaching（C，T，B）∉4NF，存在非平凡的多值依赖C→→T，且C不是候选键
  - 用投影分解法把Teaching分解为如下两个关系模式

CT（C，T）∈ 4NF

CB（C，B） ∈ 4NF

C→→T， C→→B是平凡多值依赖

4.2.8 多值依赖与4NF（19）

- 分解后Teaching关系中的几个问题可以得到解决
  （1）参考书需要在CB关系中存储一次
  （2）当某一课程增加一名任课教师时，只需要在CT关系中增加一个元组
  （3）某一门课要去掉一本参考书，只需要在CB关系中删除一个相应的元组

4.2.9 连接依赖与5NF

- 设R是一个关系模式，R的属性子集为R_1，R_2，R_3，R_4，R_5，…，当且仅当R的每个合法值都等于R_1，R_2，R_3，R_4，R_5，…，的投影连接时，称R满足连接依赖
- 设R是一个满足第五范式的关系模式，当且仅当R的每一个非平凡连接依赖都被R的候选键所蕴含。也就是说，从4NF中消除非候选键所蕴含的连接依赖

4.2.10 规范化小结（1）

- 关系数据库的规范化理论是数据库逻辑设计的工具
- 一个关系只要其分量都是不可分的数据项，它就是规范化的关系，但这只是最基本的规范化
- 规范化程度可以有6个不同的级别，即6个范式

4.2.10 规范化小结（2）

- 规范化程度过低的关系不一定能够很好地描述现实世界，可能会存在插入异常、删除异常、修改复杂、数据冗余等问题，解决的方法就是对其进行规范化，转换成高级范式
- 一个低一级范式的关系模式，通过模式分解可以转换为若干个高一级范式的关系模式集合，这种过程就叫关系模式的规范化

4.2.10 规范化小结（3）

- 关系模式规范化的基本步骤

消除决定属性集非键的非平凡函数依赖	1NF ↓ 消除非主属性对键的部分函数依赖 2NF ↓ 消除非主属性对键的传递函数依赖 3NF ↓ 消除主属性对键的部分和传递函数依赖 BCNF ↓ 消除非平凡且非函数依赖的多值依赖 4NF ↓ 消除非主属性对键的连接依赖 5NF

4.2.10 规范化小结（4）

- 规范化的基本思想
  - 消除不合适的数据依赖
  - 使各关系模式达到某种程度的“分离”
  - 采用“一事一地”的模式设计原则
    - 让一个关系描述一个概念、一个实体或者实体间的一种联系。若多于一个概念就把它“分离”出去
  - 所谓规范化实质上是概念的单一化

4.2.10 规范化小结（5）

- 不能说规范化程度越高的关系模式就越好
- 在设计数据库模式结构时，必须对现实世界的实际情况和用户应用需求作进一步分析，确定一个合适的、能够反映现实世界的模式
- 上面的规范化步骤可以在其中任何一步终止

4.2.10 规范化小结（6）

- 数据库规范化的程度越高，其关系表的数量就越多。关系表数量的增多，增加了表之间连接运算的代价，增加了I/O操作，又会影响数据库的执行速度和性能
- 通常，在工程应用中，关系模式的规范化工作仅做到3NF。当一个关系分解到3NF时，关系中不合理的属性基本消除，且因为规范化程度不是太高，数据库有较好的性能

4.3 数据依赖的公理系统（1）

- 逻辑蕴含
  - 对于满足一组函数依赖 F 的关系模式R <U，F>，其任何一个关系r，若函数依赖X→Y都成立，（即r中任意两元组t，s，若t[X]=s[X]，则t[Y]=s[Y]），则称F逻辑蕴含X →Y

4.3 数据依赖的公理系统（2）

- 4.3.1 Armstrong公理系统
- 4.3.2 闭包
- 4.3.3 函数依赖集等价与覆盖

4.3.1 Armstrong公理系统

- 关系模式R <U，F >有以下的推理规则
  - A1. 自反律（Reflexivity）：若$Y \subseteq X \subseteq U$，则$X \rightarrow Y$为F所蕴含
  - A2. 增广律（Augmentation）：若$X \rightarrow Y$为F所蕴含，且$Z \subseteq U$，则$XZ \rightarrow YZ$为F所蕴含
  - A3. 传递律（Transitivity）：若$X \rightarrow Y$及$Y \rightarrow Z$为F所蕴含，则$X \rightarrow Z$为F所蕴含

Armstrong公理系统的正确性（1）

（1）自反律：若$Y \subseteq X \subseteq U$，则$X \rightarrow Y$为F所蕴含

$$\left.\begin{array}{l}\text{若}\ t[X]=s[X] \\ \text{且}\ Y \subseteq X\end{array}\right\} \Longrightarrow t[Y]=s[Y] \Longrightarrow X \rightarrow Y$$

Armstrong公理系统的正确性（2）

（2）增广律：若$X \rightarrow Y$为F所蕴含，且$Z \subseteq U$，则$XZ \rightarrow YZ$ 为F所蕴含

$$\left.\begin{array}{l}\text{若}\ t[XZ]=s[XZ] \Longrightarrow \left\{\begin{array}{l} t[Z]=s[Z] \\ \left.\begin{array}{l} t[X]=s[X] \\ \text{又因}\ X \rightarrow Y \end{array}\right\} \Longrightarrow t[Y]=s[Y] \end{array}\right. \end{array}\right\}$$

$$\Longrightarrow t[YZ]=s[YZ] \Longrightarrow XZ \rightarrow YZ$$

Armstrong公理系统的正确性（3）

（3）传递律：若$X \rightarrow Y$及$Y \rightarrow Z$为F所蕴含，则$X \rightarrow Z$为 F所蕴含

$$\left.\begin{array}{l}\text{若}\ t[X]=s[X] \\ \text{由于}\ X \rightarrow Y\end{array}\right\} \Longrightarrow \left.\begin{array}{l} t[Y]=s[Y] \\ \text{又因}\ Y \rightarrow Z \end{array}\right\} \Longrightarrow t[Z]=s[Z]$$

$$\Longrightarrow X \rightarrow Z$$

Armstrong公理的推论

- 由Armstrong公理系统得到的三条推理规则
  - 合并规则：由$X \rightarrow Y$，$X \rightarrow Z$，有$X \rightarrow YZ$
  - 伪传递规则：由$X \rightarrow Y$，$WY \rightarrow Z$，有$XW \rightarrow Z$
  - 分解规则：由$X \rightarrow Y$及$Z \subseteq Y$，有$X \rightarrow Z$
- 根据合并规则和分解规则，可得如下引理

  $X \rightarrow A_1 A_2 \ldots A_k$成立$\Leftrightarrow X \rightarrow Ai$成立（i=1，2，…，k）

Armstrong公理系统的有效性与完备性（1）

- Armstrong公理系统是有效的，完备的
  - 有效性
    - 指由F出发根据Armstrong公理推导出来的每个函数依赖一定在F所蕴含的函数依赖的全体之中
  - 完备性
    - F所蕴含的函数依赖的全体中的每一个函数依赖，必定可以由F根据Armstrong公理导出

Armstrong公理系统的有效性与完备性（2）

- Armstrong公理系统的有效性由Armstrong公理系统的正确性得到证明，需要进一步证明Armstrong公理系统的完备性

Armstrong公理系统的完备性

- 属性集X关于函数依赖集F的闭包
  - 定义
    - 在R<U，F>中，$X \subseteq U$，$X_F^+ = \{A \mid X \rightarrow A$能由F根据Armstrong公理导出$\}$，称$X_F^+$为属性集X关于函数依赖集F的闭包
- 引理

  $X \rightarrow Y$能够由F根据Armstrong公理导出

  $\Leftrightarrow Y \subseteq X_F^+$

Armstrong公理完备性证明（1）

- Armstrong公理完备性的证明
  - 证明逆否命题，证明若$X \rightarrow Y$不能用Armstrong公理从F中导出，那么它必然不被F逻辑蕴涵。或者说，存在一个具体关系r，F中所有的函数依赖都满足r，而不能用公理推出$X \rightarrow Y$，却不满足r
  - 设$X \rightarrow Y$不能用Armstrong公理导出，并建立关系r

Armstrong公理完备性证明（2）

	X_F^+	$U - X_F^+$
t	11…1	00 …0
s	11…1	11…1

- 公理完备性需证明

  （1）在r中F的所有函数依赖都成立

  （2）在r中，$X \rightarrow Y$不能成立

Armstrong公理完备性证明（3）

（1）设$V \rightarrow W$是F中任一函数依赖，则有下列两种情况

a）$V \subseteq X_F^+$。因为$V \subseteq X_F^+$，所以有$X \rightarrow V$；于是$X \rightarrow W$成立，所以$W \subseteq X_F^+$。因为r中X_F^+的值全相等，所以$V \rightarrow W$在r上成立

b）$V \not\subseteq X_F^+$。如果V不完全属于X_F^+，则V在两元组t和s上的属性值必不相等，则$V \rightarrow W$在r上成立

因此，在关系r中，

F的任一函数依赖都成立

	X_F^+	$U - X_F^+$
t	11…1	00 …0
s	11…1	11…1

Armstrong公理完备性证明（4）

（2）因为X→Y不能用公理从F推出，则Y ⊆X_F^+，而X ⊆X_F^+，那么r中元组t，s在X上的值相等，而在Y上的值不等，则X→Y在r上不成立

- 结论：凡不能用公理推出的函数依赖都不被F逻辑蕴涵，即凡是F逻辑蕴涵的函数依赖都能用Armstrong公理从F导出
  - Armstrong公理是完备的

	X_F^+	$U-X_F^+$
t	11…1	00 …0
s	11…1	11…1

4.3.2 闭包（1）

- 定义
  - 设关系模式R（U），F为R上成立的FD集，则称所有用Armstrong公理从F推出的FD：X→A_i中的A_i的属性集合为X的属性集闭包，记为X^+

4.3.2 闭包（2）

- 定理
  - 设关系模式R（U）,F为R上成立的FD集，X，Y ⊆ U，则从F推出X→Y的充分必要条件：Y ⊆ X^+
- 由属性集闭包的定义可知，若想判断函数依赖X→Y是否成立，只要计算X关于函数依赖集F的闭包，若Y是X闭包中的一个元素，则X→Y

4.3.2 闭包（3）

- 算法：求属性集X关于FD集的属性集闭包X^+
  - 输入：关系模式R（U）中全部U，U上的FD集F，U的子集X
  - 输出：属性集X关于F的属性集闭包X^+

4.3.2 闭包（4）

- 方法：计算$X_{(i)}$ （i=0，1，…）
  - （1）$X_{(0)}$ =X
  - （2）$X_{(i+1)}$ =$X_{(i)}$ A，其中A是这样的属性：在F中寻找尚未用过的左边是$X_{(i)}$ 的子集的FD：Y→A，其中Y是$X_{(i)}$ 的子集
  - （3）判断$X_{(i+1)}$ =$X_{(i)}$ ，若是转（4），否则转（2）
  - （4）输出$X_{(i)}$ ，即为X^+

CS1

4.3.2 闭包（5）

- 其他表述
  - （1）令$X_{(0)}$ =X，i =0
  - （2）求B，这里B = { A |(V)(W)(V→W ∈F∧V ⊆ $X_{(i)}$ ∧A ∈ W)}；
  - （3）$X_{(i+1)}$ =B ∪$X_{(i)}$
  - （4）判断$X_{(i+1)}$ = $X_{(i)}$
  - （5）若$X_{(i+1)}$ 与$X_{(i)}$ 相等或$X_{(i)}$ =U，则$X_{(i)}$ 就是X_F^+，算法终止
  - （6）若否，则i =i +I，返回第（2）步

幻灯片 138

CS1 Chun Shan, 2024/3/25

4.3.2 闭包（6）

– 其他表述（续）
- 对X(i)中的每个元素，依次检查相应的函数依赖，将依赖它的属性加入B
- 令a_i =|$X_{(i)}$ |，{a_i }形成一个步长大于1的严格递增的序列，序列的上界是 | U |，因此该算法最多 |U| - |X| 次循环就会终止

示例

- R< U，F >，U = （A，B，C，D，E），F = {AB→C，B→D，C→E，CE→B，AC→B}，计算（AB）$_F^+$

所用依赖　（AB）$_F^+$
　　AB
AB→C　ABC
B→D　ABCD
C→E　ABCDE
（AB）$_F^+$= ABCDE

课堂练习

- R< U，F >，U = （A，B，C，G，H，I），F = {A→B，A→C，CG→H，CG→I，B→H}，计算（AG）$_F^+$

参考答案

所用依赖　（AG）$_F^+$
　　AG
A→B　AGB
A→C　AGBC
CG→H　AGBCH
CG→I　AGBCHI
（AG）$_F^+$= AGBCHI

课堂练习

- R< U，F >，U = （A，B，C，D，E，G），F = {A→E，BE→AG，CE→A，G→D}，计算（AB）$_F^+$

参考答案

所用依赖（AB）$_F^+$
　　AB
A→E　ABE
BE→AG　ABEG
G→D　ABEGD
（AB）$_F^+$= ABEGD

4.3.3 函数依赖集等价与覆盖

- 函数依赖集等价
  - 函数依赖集F，G，若$F^+ = G^+$，则称F与G等价
  - 如果F和G等价，则称F覆盖G，同时G也覆盖F
  - $F^+ = G^+ \Leftrightarrow F \subseteq G^+$，$G \subseteq F^+$

函数依赖集的最小依赖集

- 最小依赖集
  - 定义：若函数依赖集F满足下列条件，则称F为一个极小函数依赖集，也称为最小依赖集或最小覆盖
    - F中任一函数依赖$X \rightarrow A$，A必是单属性。（右部单属性化）
    - F中不存在这样的函数依赖$X \rightarrow A$，使得F与$F - \{X \rightarrow A\}$等价（没有多余的FD）
    - F中不存在这样的函数依赖$X \rightarrow A$，在X中有真子集Z，使得F与$F - \{X \rightarrow A\} \cup \{Z \rightarrow A\}$等价（每个FD左部没有多余属性）

函数依赖集F的极小化处理

- 函数依赖集F的极小化处理
  - 定理：每个函数依赖集F均等价于一个极小函数依赖集F_m，此F_m为F的最小依赖集

F的极小化算法

- 逐个检查F中各函数依赖FD_i：$X \rightarrow Y$，若$Y=A_1A_2 \ldots A_k$，$k \geq 2$，则用$\{X \rightarrow A_i \mid I=1,2,\ldots k\}$ 代替$X \rightarrow Y$
- 逐个检查F中各函数依赖$X \rightarrow A$，设$X = B_1 \ldots B_m$，逐个考查B_i，若$A \in (X-B_i)_F^+$，则以$(X - B_i)$取代X。直到F不再改变
- 逐个检查F中各函数依赖$X \rightarrow A$，令$G = F - \{X \rightarrow A\}$，若$A \in (X)_G^+$，则从F中去掉该函数依赖，直到F不再改变

示例

- $F = \{A \rightarrow B, B \rightarrow A, A \rightarrow C, B \rightarrow C\}$，求$F_m$
- 参考答案

  $F_m = \{A \rightarrow B, B \rightarrow A, B \rightarrow C\}$

  或者

  $F_m = \{A \rightarrow B, B \rightarrow A, A \rightarrow C\}$

求解过程（1）

- 右部单属性化
  - 当前的每个FD右部已经单属性化
- 每个FD左部没有多余属性
  - 当前的每个FD左部已经没有多余属性
- 没有多余的FD
  - 对$A \rightarrow B$和$B \rightarrow C$使用传递规则推导出$A \rightarrow C$。因此$A \rightarrow C$是冗余的，可以删除
  - 对$B \rightarrow A$和$A \rightarrow C$使用传递规则推导出$B \rightarrow C$。因此$B \rightarrow C$是冗余的，可以删除

求解过程（2）

- F_m= {A→B，B→A，B→C}

或

- F_m= {A→B，B→A，A→C}

课堂练习

- F = {C→A，A→G，CG→B，B→A}，求 F_m

参考答案

- F_m= {A→G，C→B，B→A}

求解过程（1）

- 考查每个FD右部是否满足单属性化
  - 当前的每个FD右部已经单属性化
- 考查每个FD左部是否还存在多余属性
  - 因为C→A和A→G，根据传递规则得到C→G。两边应用增广律，有CC→CG
  - 又因CC→CG和CG→B，根据传递规则得到CC→B，即C→B
  - 得到等价集合F'：{C→A，A→G，C→B，B→A}

求解过程（2）

- 考查FD属性集中是否还存在多余的FD
  - 对C→B和B→A使用传递规则，推导出C→A，因此C→A在F'中是冗余的，可以删除
- F_m={A→G，C→B，B→A}

课堂练习

- S<U，F>，U={S#，SD，MN，CN，G}，F={S#→SD，S#→MN，SD→MN，（S#，CN）→G，（S#,SD）→SD}，求F_m

参考答案

- F_m={S#→SD，SD→MN，（S#,CN）→G}

求解过程（1）

- 考查每个FD右部是否满足单属性化
  - 当前的每个FD右部已经单属性化
- 考查每个FD左部是否还存在多余属性
  - 因有S#→SD和（S#，SD）→SD，所以（S#，SD）→SD的左部存在多余属性，去掉多余属性SD，得到S#→SD
  - 得到等价集合F'：{S#→SD，S#→MN，SD→MN，（S#，CN）→G}

求解过程（2）

- 考查FD集中是否还存在多余的FD
  - 对S#→SD 和SD→MN使用传递规则，推导出S#→MN，因此S#→MN在F'中是冗余的，可以删除
- F_m={S#→SD，SD→MN，（S#，CN）→G}

4.4 模式的分解（1）

- 把低一级的关系模式分解为若干个高一级的关系模式的方法不是唯一的
- 只有能够保证分解后的关系模式与原关系模式等价，分解方法才有意义

4.4 模式的分解（2）

- 三种模式分解等价的定义
  1. 分解具有无损连接性
  2. 分解要保持函数依赖
  3. 分解既要保持函数依赖，又要具有无损连接性

4.4 模式的分解（3）

- 关系模式R<U,F>的一个分解
  $\rho=\{R_1<U_1, F_1>, R_2<U_2, F_2>, \ldots, R_n<U_n, F_n>\}$
  其中$U=\bigcup_{i=1}^{n} U_i$，且不存在 $U_i \subseteq U_j$，F_i为 F在 U_i 上的投影
- 函数依赖集合$\{X \rightarrow Y \mid X \rightarrow Y \in F^+ \wedge XY \subseteq U_i\}$的一个覆盖 F_i 叫作 F 在属性 U_i 上的投影

4.4 模式的分解（4）

例：SL（Sno， Sdept， Sloc）
F={ Sno→Sdept，Sdept→Sloc，Sno→Sloc}
SL∈2NF（每一个非主属性都完全函数依赖于键）

分解方法可以有多种：
1. SL分解为三个关系模式： SN（Sno）
SD（Sdept）
SO（Sloc）
2. SL分解为下面二个关系模式： NL（Sno，Sloc）
DL（Sdept，Sloc）
3. SL分解为下面二个关系模式： ND（Sno，Sdept）
NL（Sno，Sloc）

4.4 模式的分解（5）

- 具有无损连接性的模式分解
  关系模式R<U，F>的一个分解 ρ={ R_1<U_1，F_1>，R_2<U_2，F_2>， …，R_n<U_n，F_n>}
  若R与R_1、R_2、…、R_n自然连接的结果相等，则称关系模式R的这个分解ρ具有无损连接性（Lossless join）
- 具有无损连接性的分解保证不丢失信息。
- 无损连接性不一定能解决插入异常、删除异常、修改复杂、数据冗余等问题

4.4 模式的分解（6）

例：
SL（Sno， Sdept， Sloc）
F={ Sno→Sdept，Sdept→Sloc，Sno→Sloc}
- 第3种分解方法具有无损连接性
  ND（Sno，Sdept）
  NL（Sno，Sloc）
  - 问题
    - 这种分解方法没有保持原关系中的函数依赖
    - SL中的函数依赖Sdept→Sloc没有投影到关系模式ND、NL上

4.4 模式的分解（7）

- 保持函数依赖的模式分解
  - 设关系模式R<U,F>被分解为若干个关系模式 R_1<U_1,F_1>，R_2<U_2,F_2>，…，R_n<U_n,F_n>（其中U=U_1∪U_2∪…∪U_n，且不存在U_i ⊆ U_j，F_i为F在U_i上的投影），若F所逻辑蕴含的函数依赖一定也由分解得到的某个关系模式中的函数依赖F_i所逻辑蕴含，则称关系模式R的这个分解是保持函数依赖的（Preserve dependency）

4.4 模式的分解（8）

例：
SL（Sno， Sdept， Sloc）
F={ Sno→Sdept，Sdept→Sloc，Sno→Sloc}

4. 将SL分解为下面二个关系模式
ND（Sno，Sdept）
DL（Sdept, Sloc）
这种分解方法保持了函数依赖

4.4 模式的分解（9）

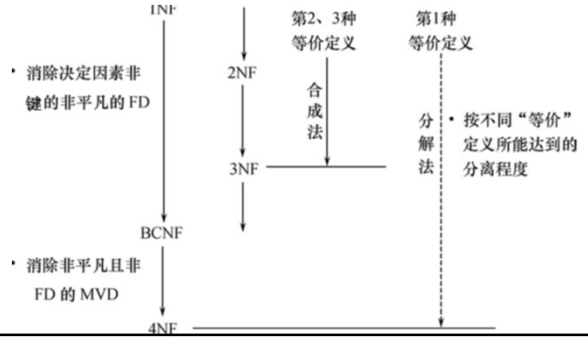
- 如果一个分解具有无损连接性，则它能够保证不丢失信息
- 如果一个分解保持了函数依赖，则它可以减轻或解决各种异常情况
- 分解具有无损连接性和分解保持函数依赖是两个互相独立的标准。具有无损连接性的分解不一定能够保持函数依赖；同样，保持函数依赖的分解也不一定具有无损连接性

4.4 模式的分解（10）

- 第1种分解方法既不具有无损连接性，也未保持函数依赖，它不是原关系模式的一个等价分解
- 第2种分解方法既没有保持函数依赖，也不具有无损连接性
- 第3种分解方法具有无损连接性，但未持函数依赖
- 第4种分解方法既具有无损连接性，又保持了函数依赖

4.5 小结（1）

- 关系模式的规范化，其基本思想



4.5 小结（2）

- 若要求分解具有无损连接性，那么模式分解一定能够达到4NF
- 若要求分解保持函数依赖，那么模式分解一定能够达到3NF，但不一定能够达到BCNF
- 若要求分解既具有无损连接性，又保持函数依赖，则模式分解一定能够达到3NF，但不一定能够达到BCNF

4.5 小结（3）

- 规范化理论为数据库设计提供了理论的指南和工具
  - 也仅仅是指南和工具
- 并不是规范化程度越高，模式就越好
  - 必须结合应用环境和现实世界的具体情况合理地选择数据库模式

参考资料（1）

- 第六章，王珊 萨师煊，数据库系统概论（第5版）， 北京：高等教育出版社，2014
- 第10章，周志逵 郭贵锁等编著，数据库系统原理，北京：清华大学出版社，2008
- Chapter 10, Elmasri & Navathe, Fundamentals of Database Systems, 5th edition, Addison-Wesley, 2006

参考资料（2）

- 邵佩英 徐俊刚 王文杰等译，[美]Ramez Elmasri & Shamkant B. Navathe著，数据库系统基础初级篇（第5版），北京：人民邮电出版社，2007

Thank You